

BEE 4750/5750

Lab 1 Solutions

2026-09-22

i Note

Labs are graded 0–2 on effort toward demonstrated work, not on correctness. The notes in this key describe what a complete answer might look like.

Setup

The model setup is as given in the lab.

```
landfill_params = let
    k = 0.15          # generation-response rate          [1/yr]
    L0 = 100.0        # methane generation potential      [m³ CH₄/Mg]
    Wmax = 1.0e5      # waste acceptance rate while open  [Mg/yr]
    active_life = 20.0 # years the landfill accepts waste  [yr]
    (; k, L0, Wmax, active_life)
end

T_total = 40.0      # total simulation horizon [yr]
G0 = 0.0            # initial gas generation rate [m³ CH₄/yr]

# Waste acceptance rate: constant while the landfill is open, zero after closure.
function waste_rate(t, p)
    if t < p.active_life
        return p.Wmax
    else
        return 0.0
    end
end
```

Problem 1: Discretize and Implement

Derivation

Approximate the derivative at time t with a forward difference over one step:

$$\frac{dG}{dt} \approx \frac{G(t + \Delta t) - G(t)}{\Delta t}.$$

Forward Euler evaluates the right-hand side at the *start* of the step, using what we already know at time t :

$$\frac{G(t + \Delta t) - G(t)}{\Delta t} = k(L_0 W(t) - G(t)).$$

Solving for the unknown:

$$G(t + \Delta t) = G(t) + \Delta t k(L_0 W(t) - G(t)).$$

We can consolidate:

$$G(t + \Delta t) = (1 - k\Delta t)G(t) + k\Delta t L_0 W(t).$$

Each step blends the current gas generation rate with the rate the waste in place would eventually produce. $L_0 W(t)$, and $k\Delta t$ determine the influence of new waste.

i Note

Evaluating W or G at $t + \Delta t$ on the right-hand side is wrong (that is no longer forward Euler). For forward Euler, the **present state** (at time t) is what determines the update rule.

Implementation

The only line to fill in is the update, which is the derivation above written in code.

```
landfill_gas_rate(G, t, p) = p.k * (p.L0 * waste_rate(t, p) - G)

function landfill_gas_simulate(G_ic, T, Δt, p)
    steps = Int(round(T / Δt))
    G = zeros(steps + 1) # index 1 holds the initial condition
    G[1] = G_ic
    for i in 1:steps
        t = (i - 1) * Δt
        G[i+1] = G[i] + Δt * landfill_gas_rate(G[i], t, p)
    end
    return G
end
```

The sanity check:

```
Δt_check = 0.5
G_check = landfill_gas_simulate(G0, T_total, Δt_check, landfill_params)
times_check = collect(0:length(G_check)-1) .* Δt_check

p_check = plot(times_check, G_check ./ 1e6, color=:blue, legend=false,
    xlabel="Time [yr]", ylabel="Gas generation [million m³/yr]")
vline!(p_check, [landfill_params.active_life], color=:black, linewidth=2,
    linestyle=:dash)
plot!(p_check, size=(900, 380), left_margin=10mm, bottom_margin=10mm)
```

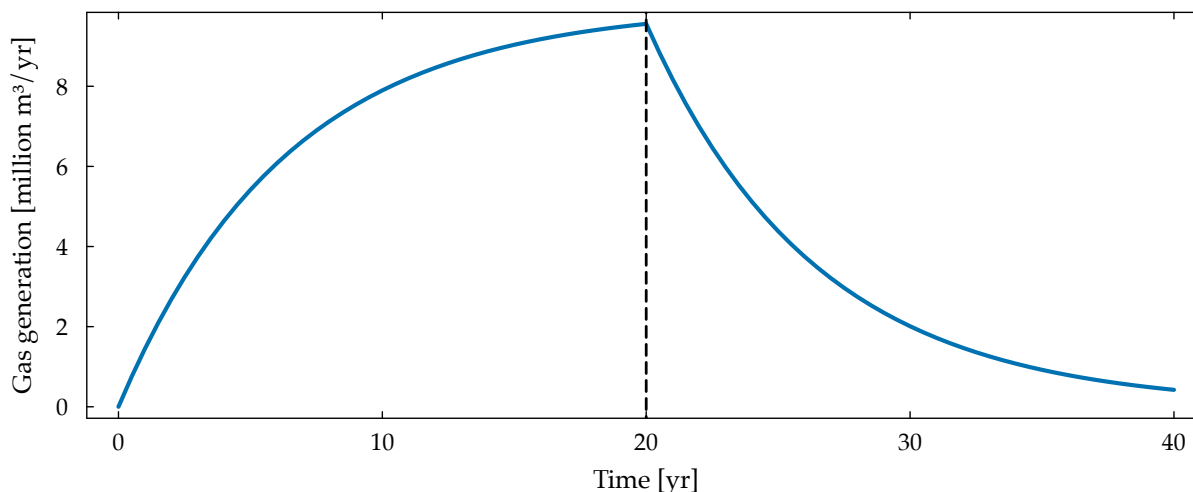


Figure 1: Methane generation at a step of half a year. The dashed line marks closure.

This shape makes sense given what the model tells us. While the landfill is open (from $t = 0$ to $t = 20$), G rises quickly at first and then flattens as it approaches $L_0 W_{\max} = 10,000,000 \text{ m}^3 \text{ CH}_4/\text{yr}$, the rate that constant waste acceptance would eventually support. It never gets there: by facility closure it has reached 96% of that level. The peak falls exactly at closure, because that is the moment the source switches off. After closure, G decays exponentially on the same timescale it rose, down to 4% of the peak by year 40.

Problem 2: How Small Does Δt Need to Be?

Problem 2a: Characteristic Timescale

The model has one rate, $k = 0.15 \text{ yr}^{-1}$, so the characteristic timescale is

$$\frac{1}{k} = \frac{1}{0.15 \text{ yr}^{-1}} \approx 6.7 \text{ yr}$$

The waste acceptance rate $W(t)$ is a forcing, not a rate. It switches off abruptly at closure, but G still responds to that switch on the $1/k$ timescale.

For a starting step, we might pick $\Delta t = 0.5 \text{ yr}$. That is about thirteen steps per timescale, which is well inside it, and it divides 20 evenly, so a grid point lands exactly on closure. But any value reasonably below the timescale is defensible as a starting point.

i Impact of choosing too large an initial step

Look at the rearranged update rule from Problem 1. The weight on $G(t)$ is $1 - k\Delta t$. Once Δt exceeds $1/k$, that weight is negative: after closure, each step flips the sign of G , and the model reports *negative* gas generation, which is physically impossible. Beyond $2/k \approx 13 \text{ yr}$ the flips grow and the solution blows up. So $1/k$ is a hard ceiling on Δt , and you usually want to pick something much smaller than that.

```
too_coarse = landfill_gas_simulate(G0, T_total, 7.0, landfill_params)
most_negative = minimum(too_coarse)
```

At $\Delta t = 7 \text{ yr}$, just past the ceiling, the lowest value of G is $-500,062 \text{ m}^3 \text{ CH}_4/\text{yr}$.

Problem 2b: Convergence Results

Let's build the reference. A step of 10^{-4} yr is 400,000 steps, which still runs in a few milliseconds for this simple model.

```
Δt_ref = 1e-4
G_ref = landfill_gas_simulate(G0, T_total, Δt_ref, landfill_params)
peak_ref = maximum(G_ref)
```

The reference peak is 9,502,141 m³ CH₄/yr.

i Checking the Reference

You did not have to do this, but here we can actually find an analytic solution to the maximum gas generation to compare the reference to (this is not possible in general). While the landfill is open, $G(t) = L_0 W_{\max}(1 - e^{-kt})$, so the true peak at closure is

$$L_0 W_{\max}(1 - e^{-k \cdot 20}) = 10^7(1 - e^{-3}),$$

which is 9,502,129 m³ CH₄/yr.

The reference is off by 11 m³ CH₄/yr, far smaller than any error in the table below, so it is safe to measure against. Again, if you don't have an analytic solution, just pick something that seems very small.

Starting from the Problem 2a estimate and halving four times:

```
Δts = [0.5, 0.25, 0.125, 0.0625, 0.03125]
peaks = zeros(length(Δts))
errs = zeros(length(Δts))
for i in 1:length(Δts)
    G_test = landfill_gas_simulate(G0, T_total, Δts[i], landfill_params)
    peaks[i] = maximum(G_test)
    errs[i] = abs(peaks[i] - peak_ref)
end

orders = zeros(length(Δts) - 1)
for i in 1:length(Δts)-1
    orders[i] = log2(errs[i] / errs[i+1])
end
```

Δt [yr]	Peak G [m ³ CH ₄ /yr]	Error vs. reference	Error improvement
0.5	9,557,749	55,608	—
0.25	9,530,042	27,901	0.99
0.125	9,516,109	13,969	1.00
0.0625	9,509,125	6,985	1.00
0.03125	9,505,629	3,488	1.00

```

error_ticks = [5e3, 1e4, 2e4, 5e4]
p_conv = plot(Δts, errs, xscale=:log10, yscale=:log10, markershape=:circle,
    markersize=7, color=cb_vermillion, label="forward Euler",
    xticks=(Δts, string.(Δts)), yticks=(error_ticks, with_commas.(error_ticks)),
    xlabel="Δt [yr]", ylabel="Error in peak [m³/yr]", legend=:bottomright)
# Slope-1 reference: errors parallel to it mean a first-order method.
plot!(p_conv, Δts, errs[1] .* (Δts ./ Δts[1]), color=:black, linewidth=2,
    linestyle=:dash, label="slope 1")
plot!(p_conv, size=(900, 400), left_margin=12mm, bottom_margin=10mm)

```

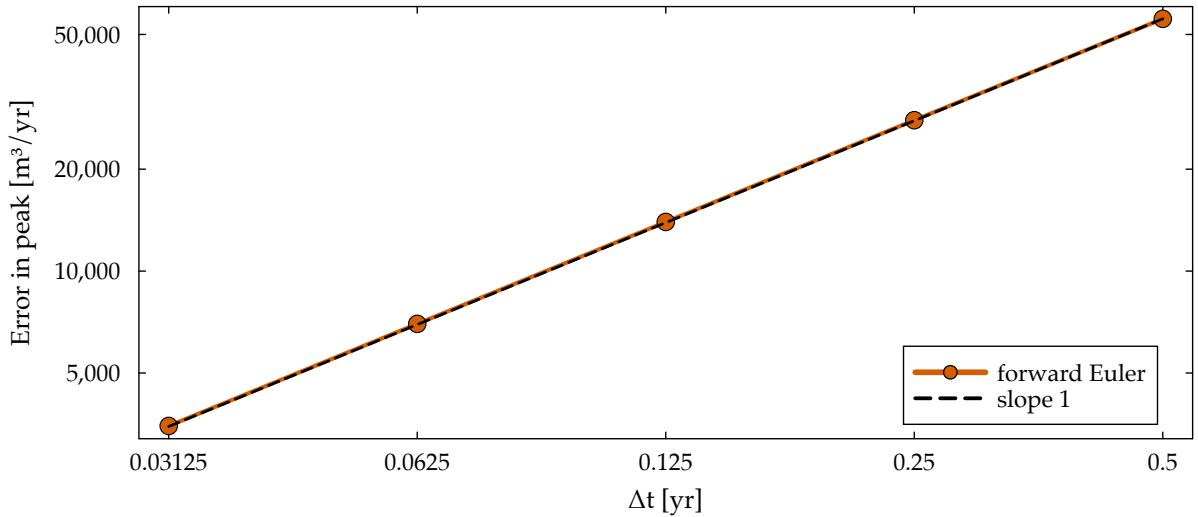


Figure 2: Error in the peak methane generation rate against step size, on log-log axes.

The error halves every time Δt halves: each improvement is close to 1, and the errors run parallel to the slope-1 line. That is first-order convergence, which is what the Taylor argument predicts. Forward Euler keeps the first two terms of $G(t + \Delta t) = G(t) + \Delta t G'(t) + \frac{1}{2} \Delta t^2 G''(t) + \dots$, so each step commits an error proportional to Δt^2 . Reaching closure takes $20/\Delta t$ steps, and $20/\Delta t$ errors of size Δt^2 add up to an error proportional to Δt .

The errors also all have the same sign: forward Euler *over*-predicts the peak at every step size. While the landfill fills, dG/dt shrinks over each step as G climbs toward $L_0 W_{\max}$. Forward Euler uses the rate from the start of the step, when it is largest, so it overshoots a little on every step.

i Some Pathologies Unrelated to Coding

You might see the following, but these are not code errors:

- **Improvements that wobble around 1.** If you start from $1/(10k)$ and rounded it to 0.67 yr get a sequence whose grid points never land on closure. The step that straddles year 20 keeps adding waste for part of a step, and that fraction changes irregularly as Δt halves. Starting from 0.67, the improvements run 0.87, 0.77, 1.30, 0.84, while the slope across the whole sequence is 0.95. This is still first order.
- **Improvements that creep above 1 at the fine end.** This one *is* a problem, with the reference rather than the method: Δt_{ref} was not much finer than the finest test step. The reference carries an error of the same sign, so the measured errors shrink faster than the true ones. With $\Delta t_{\text{ref}} = 0.01$ yr and the step sizes above, the improvements run 1.02, 1.06, 1.13, 1.30. The fix is a finer reference.

Problem 3: Make and Justify a Recommendation

From this analysis, we might recommend using $\Delta t = 0.25$ yr, a quarterly step.

Other step sizes are equally defensible. Even an annual step puts the peak only 1.2% high, which is a real feature of this model: a timescale of nearly seven years is forgiving.